Problem 1:

```
[1]: #to calculate the area of circle
     a=int(input("Enter the radius"))
     area=3.14*(a^2)
     print(area)

     Enter the radius 4
     18.84
```

Problem 2:

```
[3]: #voting system
     a=int(input("Enter age"))
     if (a>=18):
         print("you can vote")
     else:
         print("You cannot vote")

     Enter age 12
     You cannot vote
```

Problem 3:

```
•[12]: #to check for vote and car
       a=int(input("Enter age"))
       b=input("Enter y or n for yes and no to license")
       if(a>=18):
           if(b=="y" or b=="Y"):
               print("Eligible for license and you can drive a car")
           else:
               print("Eligible for vote but you do not have license")
       else:
           print("Not eligible for License and you cannot drive a car")

       Enter age 56
       Enter y or n for yes and no to license n
       Eligible for vote but you do not have license
```

Problem 4:

[5]:
```
#marks system
a=int(input("Enter marks"))
if(a>=90):
    print("A Grade")
elif(a>=70):
    print("B Grade")
else:
    print("C and lower Grade")
```

```
Enter marks 45
C and lower Grade
```

Problem 5:

[14]:
```
#days printing
a=int(input("Enter from 1 to 7: "))
if(a==1):
    print("Monday")
elif(a==2):
    print("Tuesday")
elif(a==3):
    print("Wednesday")
elif(a==4):
    print("Thursday")
elif(a==5):
    print("Friday")
elif(a==6):
    print("Saturday")
elif(a==7):
    print("Sunday")
else:
    print("Invalid number")
```

```
Enter from 1 to 7:  4
Thursday
```

Problem 6:

[16]:
```
#days printing with match innit
a=int(input("Enter from 1 to 7: "))
match a:
    case 1:
        print("Monday")
    case 2:
        print("Tuesday")
    case 3:
        print("Wednesday")
    case 4:
        print("Thursday")
    case 5:
        print("Friday")
    case 6:
        print("Saturday")
    case 7:
        print("Sunday")
    case _:
        print("Invalid")
```

```
Enter from 1 to 7:  4
Thursday
```

Problem 7:

[20]:
```
a=input("Enter an alphabet")
if(a=="a" or "e" or "i" or "o" or "u" or "A" or "E" or "I" or "O" or "U"):
    print("It is a vowel")
else:
    print("It is a consonant")
```

```
Enter an alphabet E
It is a vowel
```

Problem 8:

1]:
```
a=100
b=5
if (a>b and a>=100):
    print("a is greater than b and more than 100")
else:
    print("not greater or more than 100")
```

```
a is greater than b and more than 100
```

Problem 9:

```
a=100
if not a:
    print("a is greater than b and more than 100")
else:
    print("a has value")
```

Problem 1:

```
[2]: #fah
for i in range(5,10):
    print (i)
```

```
5
6
7
8
9
```

Problem 2:

```
[3]: for i in range(1,10,2):
    print(i)
```

```
1
3
5
7
9
```

Problem 3:

```
[4]: for i in range(2):
    for j in range(2):
        print(i,j,end="\t") #\t is just there for space so you can put " " as well
    print()
```

```
0 0     0 1
1 0     1 1
```

Problem 4:

[15]:
```
a=[[0,0],[0,0]]
for i in range(2):
    for j in range(2):
        a[i][j]=int(input())
    print()
for i in range(2):
    for j in range(2):
        print(a[i][j],end=" ")
    print()
```

```
1
3

4
6

1 3
4 6
```

Problem 5:

```
#Star pro
for i in range(1,7):
    for j in range(i):
        print("*", end=" ")
    print()
```

```
*
* *
* * *
* * * *
* * * * *
* * * * * *
```

Problem 5.1:

```
#factorial in for loop
n = int(input("enter factorial"))
fact=1
if n<0:
    print("invalid")
elif n==0:
    print("1")
else:
    for i in range(1, n+1):
        fact = fact*i
    print(fact)
```

```
enter factorial 3
6
```

Problem 6, 6.1, 6.2:

```
[22]: def add(a,b):
          return a+b
      add(5,6)
```

```
[22]: 11
```

```
[23]: def arith(a,b):
          return a+b,a-b,a*b
      arith(5,6)
```

```
[23]: (11, -1, 30)
```

```
[24]: def arith(a,b):
          return a+b
```

```
[25]: arith(4,5)
```

```
[25]: 9
```

Problem 7:

```
]: #function using swapping
   def swap(a,b):
       return b,a
   swap(5,6)
```

```
]: (6, 5)
```

Problem 8:

```
: def factorial(n):
      if n==0 or n==1:
          return 1
      else:
          return n*factorial(n-1)
  num=int(input("enter a number"))
  print(factorial(num))
```

```
enter a number 6
720
```

Problem 9:

```
def fib(n):
    f=0
    s=1
    print(f,s)
    for i in range(n+1):
        t=f+s
        print(t)
        f=s
        s=t
    fib(5)
```

Problem 10:

```
0]: #while loop
    i=1
    while i<=5:
        print(i)
        i += 1
```

```
1
2
3
4
5
```

Lab Practical 2

Q: To print factorial using recursion

Input:

```
#factorial using recursion
def factorial(n):
    if n==0 or n==1:
        return 1
    else:
        return n*factorial(n-1)
num=int(input("enter a number"))
print(factorial(num))
```

Output:

```
enter a number 6
720
```

Q: To print the word count of a given sentence

Input:

```
#Lab 2
sentence = "hello world"
count=0
if sentence != " ":
    count = 1
for i in sentence:
    if i == " ":
        count = count+1
print("number of words:")
print(count)
```

Output:

```
number of words:
2
```

Q: To print the Fibonacci series using recursive function
Input:

```
#fibonacci using recursion
def fib(n):
    if n <= 1:
        return n
    return fib(n - 1) + fib(n - 2)
a=int(input("enter a number"))
print(fib(a))
```

Output:

```
enter a number 4
3
```

Lab Practicals
Q: To create a simple calculator
Input:

```
#---------------------------- 11th Feb Lab
#simple calculator
a = int(input("Enter a number"))
b = int(input("Enter 2nd number"))
c = int(input("Enter 1 for addition, 2 for subtraction, 3 for multiplication, 4 for division"))
add = a+b
sub = a-b
mul = a*b
div = a/b
if(c==1):
    print(add)
elif(c==2):
    print(sub)
elif(c==3):
    print(mul)
elif(c==4):
    print(div)
else:
    print("invalid")
```

Output:

```
Enter a number 34
Enter 2nd number 67
Enter 1 for addition, 2 for subtraction, 3 for multiplication, 4 for division 2
-33
```

Q:Temperature Conversion
i) Fahrenheit to Celcius
Input:

```
#c/5=f-32/9
#from f to c
f=int(input("Enter temperature in Fahrenheit"))
c=(5*(f-32))/9
print(c)
```

Output:

```
Enter temperature in Fahrenheit 98
36.666666666666664
```

ii)Celsius to Fahrenheit
Input:

```
#c/5=f-32/9
#from c to f
c=float(input("Enter temperature in Celcius"))
f=((9*c)/5)+32
print(f)
```

Output:

```
Enter temperature in Celcius 36.666666666666664
98.0
```

Q: Find the largest among the three numbers
Input:

```
#largest and smallest
n1=int(input("Enter a number"))
n2=int(input("Enter another number"))
n3=int(input("Enter another number"))
if(n1>n2 and n1>n3):
    print(f"{n1} is the largest among two")
elif(n2>n1 and n2>n3):
    print(f"{n2} is the largest among two")
elif(n3>n1 and n3>n2):
    print(f"{n3} is the largest among two")
else:
    print("invalid")
```

Output:

```
Enter a number 56
Enter another number 67
Enter another number 89
89 is the largest among two
```

Q:Find the smallest among the three numbers
Input:

```
#largest and smallest
n1=int(input("Enter a number"))
n2=int(input("Enter another number"))
n3=int(input("Enter another number"))
if(n1<n2 and n1<n3):
    print(f"{n1} is the smallest among two")
elif(n2<n1 and n2<n3):
    print(f"{n2} is the smallest among two")
elif(n3<n1 and n3<n2):
    print(f"{n3} is the smallest among two")
else:
    print("invalid")
```

Output:

```
Enter a number 45
Enter another number 67
Enter another number 78
45 is the smallest among two
```

Lab Practical 2
Q: To print factorial using recursion

Input:

```
#factorial using recursion
def factorial(n):
    if n==0 or n==1:
        return 1
    else:
        return n*factorial(n-1)
num=int(input("enter a number"))
print(factorial(num))
```

Output:

```
enter a number 6
720
```

Q: To print the word count of a given sentence
Input:

```
#Lab 2
sentence = "hello world"
count=0
if sentence != " ":
    count = 1
for i in sentence:
    if i == " ":
        count = count+1
print("number of words:")
print(count)
```

Output:

```
number of words:
2
```

Q: To print the Fibonacci series using recursive function
Input:

```
#fibonacci using recursion
def fib(n):
    if n <= 1:
        return n
    return fib(n - 1) + fib(n - 2)
a=int(input("enter a number"))
print(fib(a))
```

Output:

```
enter a number 4
3
```

Lab Practical 3

Prac 1: Lists and usage of various commands in list.

i) append list, and property of list

Input:

```
#list
a=[10,20,30,40]
b=[10,"anu",10.2,True]
#mutable
#multiple data items
a.append(100)
```

Output:

```
print(a)

[10, 20, 30, 40, 100]
```

ii) append and copy

Input:

```
#list
a=[10,20,30,40]
b=[10,"anu",10.2,True]
#mutable
#multiple data items
a.append(100)
c=a.copy()
print(c)
```

Output:

```
[10, 20, 30, 40, 100]
```

iii) extend, index, insert, pop
Input:

```
#list
a=[10,20,30,40]
b=[10,"anu",10.2,True]
#mutable
#multiple data items
a.append(100)
print(a.count(10)) #how many times the number appears in the list
a.extend([200,300,400])
print(a)
a.index(100)
a.insert(5,150) #index,value so 5th position and value 150
a.pop() #last element
a.pop(1) #index of element 1
```

Output:

```
1
[10, 20, 30, 40, 100, 200, 300, 400]
20
```

iv) reverse and sort
Input:

```
#list
a=[10,20,30,40]
b=[10,"anu",10.2,True]
#mutable
#multiple data items
a.append(100)
print(a.count(10)) #how many times the number appears in the list
a.extend([200,300,400])
print(a)
print(a.pop()) #last element
print(a.pop(1)) #index of element 1
print(f"{a} the 400 and 20 gets popped or removed so its not there")
a.reverse()
print(a)
a.sort() #sorts in ascending order
print(a)
```

Output:

```
1
[10, 20, 30, 40, 100, 200, 300, 400]
400
20
[10, 30, 40, 100, 200, 300] the 400 and 20 gets popped or removed so its not there
[300, 200, 100, 40, 30, 10]
[10, 30, 40, 100, 200, 300]
```

v) slicing

Input:

```
#slicing
a=[10,20,30,40,50,59,28]
print(a)
print(a[1:4:2]) #index:length:skipping numbers
a.clear()
print(a)
```

Output:

```
[10, 20, 30, 40, 50, 59, 28]
[20, 40]
[]
```

vi) slicing with negative index
Input:

```
#slicing
a=[10,20,30,40,50,59,28]
print(a)
print(a[1:4:2]) #index:length:skipping numbers
print(a)
print(a[-1])
print(a[-1:-5]) # 3 2 1 0 -1 -2 -3... so when it checks from -1 it goes right and hence nothing is printed because -5 is Null
```

Output:

```
[10, 20, 30, 40, 50, 59, 28]
[20, 40]
[10, 20, 30, 40, 50, 59, 28]
28
[]
```

Tuples properties and usage
i) tuples and its property of homogeneity, built in functions

```
#tuples
a=(10,20,30,40)
a
```

```
(10, 20, 30, 40)
```

```
a=(10,20,30,"anu",304.423)
a
```

```
(10, 20, 30, 'anu', 304.423)
```

```
#count, index, min max, length
a=(10,20,39,59,20,590,20)
print(a.count(20))
```

```
3
```

```
a.index(39)
```

```
2
```

```
print(max(a))
```

```
590
```

```
print(min(a))
```

```
10
```

```
print(len(a))
```

```
7
```

Lab 3.2 Component:

Sets

i) Adding with sets

Input and Output:

```
a={1,2,4,53}
b={24,56,36,6,1}
a.add(7)
print(a)
```

```
{1, 2, 4, 7, 53}
```

ii) Copy with sets

Input and Output

```
c=a.copy()
print(c)
```

```
{1, 2, 4, 53, 7}
```

iii) Union, intersection, difference, symmetric difference

```
a.union(b)
```

```
{1, 2, 4, 6, 7, 24, 36, 53, 56}
```

```
a.intersection(b)
```

```
{1}
```

```
a.difference(b) #difference means the elements c
```

```
{2, 4, 7, 53}
```

```
b.difference(a) #similarly elements of b not in
```

```
{6, 24, 36, 56}
```

```
a.symmetric_difference(b)
```

```
{2, 4, 6, 7, 24, 36, 53, 56}
```

iv) Discard

```
a={1,2,3,4,5,6,7,8}
a.discard(6)
print(a)
```

```
{1, 2, 3, 4, 5, 7, 8}
```

v) Update and difference update

```
a.update([9,10])
print(a)
```

```
{1, 2, 3, 4, 5, 7, 8, 9, 10}
```

```
a={23, 45, 67, 34, 56, 34, 67}
b={24,56,56,6,1}
print(a)
a.difference_update(b) #updates the set and then the result goes to a set. the difference set becomes the a set
print(a)
```

```
{34, 67, 23, 56, 45}
{34, 67, 23, 45}
```

vi) Disjoint

```
#disjoint checks for no duplicate elements in the set
a.isdisjoint(b)
```

```
True
```

vii) Disjoint and subset checking

```
a={1,2,3}
b={1,2,3,4,5,6}
print(a.issubset(b))
print(a.isdisjoint(b)) #if there is duplicate then false, if not then true
```

```
True
False
```

vii) Combined operation program
Input:

```python
def list_operations():
    user_input = input("Enter elements of the list separated by spaces: ")
    my_list = user_input.split()

    while True:
        print("\n--- List Operations ---")
        print("1. Insert an element")
        print("2. Delete an element")
        print("3. Find an element")
        print("4. Display list")
        print("5. Exit")

        choice = int(input("Enter the choice: "))

        if choice == 1:
            element = input("Enter the element to insert: ")
            my_list.append(element)
            print(f"'{element}' added to the list.")

        elif choice == 2:
            element = input("Enter the element to delete: ")
            if element in my_list:
                my_list.remove(element)
                print(f"'{element}' deleted from the list.")
            else:
                print(f"'{element}' not found in the list.")

        elif choice == 3:
            element = input("Enter the element to find: ")
            if element in my_list:
                idx = my_list.index(element)
                print(f"'{element}' found at index {idx}.")
            else:
                print(f"'{element}' not found in the list.")

        elif choice == 4:
            print("Current List:", my_list)

        elif choice == 5:
            print("Exiting")
            break

        else:
            print("Invalid")

list_operations()
```

Output:

```
Enter elements of the list separated by spaces:  4 5 6 7 8

--- List Operations ---
1. Insert an element
2. Delete an element
3. Find an element
4. Display list
5. Exit
Enter the choice:  1
Enter the element to insert:  9
'9' added to the list.

--- List Operations ---
1. Insert an element
2. Delete an element
3. Find an element
4. Display list
5. Exit
Enter the choice:  2
Enter the element to delete:  8
'8' deleted from the list.

--- List Operations ---
1. Insert an element
2. Delete an element
3. Find an element
4. Display list
5. Exit
Enter the choice:  3
Enter the element to find:  4
'4' found at index 0.

--- List Operations ---
1. Insert an element
2. Delete an element
3. Find an element
4. Display list
5. Exit
Enter the choice:  4
Current List: ['4', '5', '6', '7', '9']

--- List Operations ---
1. Insert an element
2. Delete an element
3. Find an element
4. Display list
5. Exit
Enter the choice:  5
Exiting
```

viii) with match

```python
def list_operations():
    user_input = input("Enter elements of the list separated by spaces: ")
    my_list = user_input.split()

    while True:
        print("""\n--- List Operations ---
        1. Insert an element
        2. Delete an element
        3. Find an element
        4. Display list
        5. Exit""")

        choice = input("Enter the choice: ")

        match choice:
            case "1":
                element = input("Enter the element to insert: ")
                my_list.append(element)
                print(f"'{element}' added to the list.")

            case "2":
                element = input("Enter the element to delete: ")
                if element in my_list:
                    my_list.remove(element)
                    print(f"'{element}' deleted from the list.")
                else:
                    print(f"'{element}' not found in the list.")

            case "3":
                element = input("Enter the element to find: ")
                if element in my_list:
                    idx = my_list.index(element)
                    print(f"'{element}' found at index {idx}.")
                else:
                    print(f"'{element}' not found in the list.")

            case "4":
                print("Current List:", my_list)

            case "5":
                print("Exiting")
                break

            case _:
                print("Invalid")

list_operations()
```

Output:

```
Enter elements of the list separated by spaces:  3 4 5 6 7 8

--- List Operations ---
        1. Insert an element
        2. Delete an element
        3. Find an element
        4. Display list
        5. Exit
Enter the choice:  1
Enter the element to insert:  9
'9' added to the list.

--- List Operations ---
        1. Insert an element
        2. Delete an element
        3. Find an element
        4. Display list
        5. Exit
Enter the choice:  2
Enter the element to delete:  8
'8' deleted from the list.

--- List Operations ---
        1. Insert an element
        2. Delete an element
        3. Find an element
        4. Display list
        5. Exit
Enter the choice:  3
Enter the element to find:  5
'5' found at index 2.

--- List Operations ---
        1. Insert an element
        2. Delete an element
        3. Find an element
        4. Display list
        5. Exit
Enter the choice:  4
Current List: ['3', '4', '5', '6', '7', '9']

--- List Operations ---
        1. Insert an element
        2. Delete an element
        3. Find an element
        4. Display list
        5. Exit
Enter the choice:  5
Exiting
```

1. Using for loops in set, tuple, list

```
[2]: #----------------------------------------------Lab 3.3 20th February 2026----------------------------------------------
     #nous avons faire la dictionnaire et tuple
     t=(10,20,30,40.5,"hello")
     for i in t:
         print(i)
```

```
10
20
30
40.5
hello
```

```
[6]: t=(10,20,30,40.5,"hello")
     for i in range(len(t)):
         print(i,t[i])
```

```
0 10
1 20
2 30
3 40.5
4 hello
```

```
[5]: #set
     s={10,20,30,40.5,"hello"}
     for i in s:
         print(i)
```

```
20
40.5
hello
10
30
```

```
[ ]: s={10,20,30,40.5,"hello"}
     for i in range(len(s)):
         print(i,s[i])
         #error because unordered collection
```

```
35]: #-----------------------------list--------------------
     fruits = ["apple", "banana", "cherry"]

     for fruit in fruits:
         print(fruit)
```

```
apple
banana
cherry
```

2. Nested list in set, tuple, list and dictionary

```
]: #nested list
   nes=[[1,2],[3,4],[5,6]] #index is like 0,1 then again 0,1 because different index for different list
   print(nes) #inner index is 0,1 and o,1 but outer is 0, 1, 2 for each list so [1,2] have 0 index, [3,4] have 1 index and [5,6] have 2 index
   print(nes[0])
   print(nes[0][1])
   print(nes[1][1])
   nes[1][1]=100
   print(nes[1][1])
   print(nes)
```

```
[[1, 2], [3, 4], [5, 6]]
[1, 2]
2
4
100
[[1, 2], [3, 100], [5, 6]]
```

```
#nested tuple
nes=((1,2),(3,4),(5,6))
print(nes)
print(nes[0])
print(nes[0][1])
print(nes[2][0])
nes[2][0]=100
print(nes[2][0])
print(nes) #not possible because tuple is immutable
```

```
#set inside set or nested set is not allowed
#to use nested set use frozenset (BUT ITS NOT ALLOWED)
nes={frozenset({1,2}), frozenset({3,4})}
print(nes)
```

```
{frozenset({3, 4}), frozenset({1, 2})}
```

```
#nested dictionary
data = {
    "user": {
        "id": 1,
        "name": "Alex"
    }
}
print(data["user"]["name"])
```

```
Alex
```

3. Dictionary commands(with key and value, and using items method as well)

```
#dictionnaire avec key et value using index
d={'name':"anu","mark":60, 'age' :24}
print(d)
for key in d:
    print(key,d[key])
```

```
{'name': 'anu', 'mark': 60, 'age': 24}
name anu
mark 60
age 24
```

```
#same using items method
d={'name':"anu","mark":60, 'age' :24}
for keys,value in d.items():
    print(keys, value)
```

```
name anu
mark 60
age 24
```

```
d={'name':"anu","mark":60, 'age' :24}
print(d.keys())
print(d.values())
print(d.items())
```

```
dict_keys(['name', 'mark', 'age'])
dict_values(['anu', 60, 24])
dict_items([('name', 'anu'), ('mark', 60), ('age', 24)])
```

```
d={'name':"anu","mark":60, 'age' :24}
print(d.get('name'))
print(d.get('class'))
```

```
anu
None
```

```
d={'name':"anu"}
age=d.setdefault('age',20)
print(d)
```

```
{'name': 'anu', 'age': 20}
```

```
d={'name':"anu","mark":60, 'age' :24}
d.update({"mark":20, "name": "labubu"})
print(d)
```

```
{'name': 'labubu', 'mark': 20, 'age': 24}
```

```
data={'a':1, 'b':2, 'c':3}
data.pop('a') #pops or removes a
data.popitem() #removes c because last
print(data)
```

```
{'b': 2}
```

4. List inside tuple, list inside dictionary, list inside set

```
t=([1,2],[3,4])
print(t)

#list inside set
#s={[1,2],[3,4]} #not allowed

#list inside dictionary (As key) -- NOT ALLOWED
#d={[1,2]: "hello"}
#List inside dictionary (As value) -- ALLOWED
d = {"numbers": [1,2,3]}
print(d)
```

```
([1, 2], [3, 4])
{'numbers': [1, 2, 3]}
```

5. Tuple inside set

```
t1 = (1,2,3)
list = [t1, (4,5)]
print(list)

set={(1,2,3),(3,4,5)}
print(set)

#tuple inside dictionary allowed both as key and value
d={(1,2):"A", (3,4):"B"}
print(d)

d={"num":(1,2,3)}
print(d)
```

```
[(1, 2, 3), (4, 5)]
{(3, 4, 5), (1, 2, 3)}
{(1, 2): 'A', (3, 4): 'B'}
{'num': (1, 2, 3)}
```

6. Set inside a list, a tuple and a dictionary

```python
#set inside a list
list=[{1,2},{3,4}]
print(list)

#set inside tuple
tuple=({1,2},{3,4})
print(tuple)

#set inside dictionary as value
my_dict = {"key": {1, 2, 3}}
print(my_dict)
```

```
[{1, 2}, {3, 4}]
({1, 2}, {3, 4})
{'key': {1, 2, 3}}
```

```python
#set inside dictionary as key is not allowed (using frozenkey bruteforce)
my_dict = {frozenset({1, 2, 3}): "value"}
print(my_dict)
```

```
{frozenset({1, 2, 3}): 'value'}
```

7. Dictionary inside a list, a set, a dictionary and a tuple

```python
#dictionary inside a list
print([{"id": 1, "name": "hola"}])

#dictionary inside a tuple
print(({"status": "active", "level": 10},))

#dictionary inside dictionary
print({"user_1": {"role": "admin", "points": 500}})

#dictionary inside a set with frozenset
print({frozenset({"id": 1, "val": "A"}.items())})
```

```
[{'id': 1, 'name': 'hola'}]
({'status': 'active', 'level': 10},)
{'user_1': {'role': 'admin', 'points': 500}}
{frozenset({('val', 'A'), ('id', 1)})}
```

---

*please consider all frozensets as not allowed

8. Merging dictionary

```
]: dict_a = {"a": 1, "b": 2}
   dict_b = {"b": 3, "c": 4}

   merged = dict_a | dict_b
   print(f"dictionary 1: {dict_a}")
   print(f"dictionary 2: {dict_b}")
   print(f"Merged dictionary: {merged}")
```

```
dictionary 1: {'a': 1, 'b': 2}
dictionary 2: {'b': 3, 'c': 4}
Merged dictionary: {'a': 1, 'b': 3, 'c': 4}
```

1. Using basic array functions

```
arr=[10,20,30,40,50]
print(arr)
```

```
[10, 20, 30, 40, 50]
```

```
print(arr[0])
print(arr[2])
```

```
10
30
```

```
#Manipulation
arr[1]=200
print(arr)
arr.append(600)
print(arr)
arr.insert(2,250) #index,value
print(arr)
```

```
[10, 200, 250, 30, 40, 50, 600]
[10, 200, 250, 30, 40, 50, 600, 600]
[10, 200, 250, 250, 30, 40, 50, 600, 600]
```

2. Iteration of Array

```
]: #iteration
for i in arr:
    print(i)
print(".....")
for i in range(len(arr)):
    print(f"Index: {i}, Value: {arr[i]}")
```

```
10
200
250
250
30
40
50
600
600
.....
Index: 0, Value: 10
Index: 1, Value: 200
Index: 2, Value: 250
Index: 3, Value: 250
Index: 4, Value: 30
Index: 5, Value: 40
Index: 6, Value: 50
Index: 7, Value: 600
Index: 8, Value: 600
```

3. 2D Arrays

```
[7]: #2d array
mat=[
    [1,2,3],
    [4,5,6],
    [7,8,9]
]
print(mat)
```

```
[[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

```
15]: mat = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]

for i in range(len(mat)):
    for j in range(len(mat[i])):
        print(mat[i][j])
```

```
1
2
3
4
5
6
7
8
9
```

```
[8]: #access
print(mat[0][0])
print(mat[2][2])
```

```
1
9
```

4. Importing Array and supporting list methods

```
[9]: import array
     arr=array.array('i',[10,100,30])
     print(arr)

     array('i', [10, 100, 30])

[14]: #doesnt support all list methods
      arr.append(50)
      print(arr)
      #arr.sort() --------> doesnt work because it doesnt support all list methods
      #print(arr)
      arr=array.array('i',sorted(arr))
      print(arr)
      newarr=arr.tolist()
      print(newarr)

      array('i', [10, 30, 50, 50, 50, 50, 100, 50])
      array('i', [10, 30, 50, 50, 50, 50, 50, 100])
      [10, 30, 50, 50, 50, 50, 50, 100]

[ ]:
```

1. Introduction to numpy

```
[3]: import numpy as np
     arr1=np.array([1,2,3,4,5,6])
     print(arr1)

     [1 2 3 4 5 6]

[4]: import numpy as np
     arr1=np.array([1,2,3,4])
     print(arr1)
     arr2d=np.array([
         [1,2],[3,4]]) #if you dont give extra square bracket it shows error becayse it must have 2 to 3 tuples
     print(arr2d)

     [1 2 3 4]
     [[1 2]
      [3 4]]
```

2. Manipulation of array with numpy

```
: #manipulation
  import numpy as np
  print(arr1.shape)
  print(arr2d.shape) #size of element in each dimension, arrr1 has 1,2,3,4 so 4 elements and for 2d array 2,2 cuz 2x2 matrix

  print("****")
  print(arr1)
  print(arr1.reshape(2,2)) #reshape rows, cols

  print("***")
  print(arr1)
  print(arr1.shape) #or c=np.shape(arr1)
  print(arr1.ndim) #number of dimensions
  print(arr1.size)
  print(arr1.dtype)
  print("***")
  print(arr2d)
  print(arr2d.shape)
  print(arr2d.ndim)
  print(arr2d.size)
  print(arr2d.dtype)
```

```
print(arr2d.dtype)
```

```
(4,)
(2, 2)
****
[1 2 3 4]
[[1 2]
 [3 4]]
***
[1 2 3 4]
(4,)
1
4
int64
***
[[1 2]
 [3 4]]
(2, 2)
2
4
int64
```

3. 3d Array Manipulation

```
[15]: arr3d=np.array([
          [1,2,3],[3,4,5], [5,6,7]])
      print(arr3d[0:2])
      print("*******")
      print(arr3d[:,2])
```

```
[[1 2 3]
 [3 4 5]]
*******
[3 5 7]
```

```
[20]: #modify
      arr3d=np.array([
          [1,2,3], [5,6,7]])
      #sum
      sum=np.sum(arr3d)
      print(sum) #sum of all elements
      print("sum of coloumns item like 1+5 and 2+6 and 3+7")
      sumc = np.sum(arr3d, axis=0)
      print(sumc)
```

```
24
sum of coloumns item like 1+5 and 2+6 and 3+7
[ 6  8 10]
```

```
[21]: a=np.array([1,2,3])
      print(a+2) #adds 2 to all the elements
      print(a*3)
```

```
[3 4 5]
[3 6 9]
```

4. Arithmetic operations

```
arr3d=np.array([
    [1,2,3], [5,6,7]])
#sum
sum=np.sum(arr3d)
print(sum) #sum of all elements
print("sum of coloumns item like 1+5 and 2+6 and 3+7")
sumc = np.sum(arr3d, axis=0)
print(sumc)
```

```
24
sum of coloumns item like 1+5 and 2+6 and 3+7
[ 6  8 10]
```

```
a=np.array([1,2,3])
print(a+2) #adds 2 to all the elements
print(a*3)
```

```
[3 4 5]
[3 6 9]
```

```
a=[1,6,5]
print(np.sum(a))
print(np.mean(a))
print(np.median(a)) #sort and then median like add 2 middle elements /2 incase even median
print(np.max(a))
print(np.min(a))
```

```
12
4.0
5.0
6
1
```

5. Flattening, adding and removing elements

```
a = np.array([1, 2, 3, 4, 5, 6])
b = a.reshape(2, 3)
f = b.flatten()

c = np.append(f, 7)
d = np.delete(c, 0)

print(d)
```

```
[2 3 4 5 6 7]
```

6. Linespace, arranging, printing ones

```
o = np.ones((2, 2))
e = np.eye(3)
a = np.arange(0, 10, 2)
l = np.linspace(0, 1, 5)

print(o, "\n", e, "\n", a, "\n", l)
```

```
[[1. 1.]
 [1. 1.]]
 [[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
 [0 2 4 6 8]
 [0.   0.25 0.5  0.75 1.  ]
```

i) Array addition and algorithms

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
r1 = a+b
r2 = a*b
print(r1)
print(r2)
```

```
[5 7 9]
[ 4 10 18]
```

ii)Array Transpose

```
import numpy as np
m=[[1,2,3],[4,5,6]]
mT=np.transpose(m)
print(mT)
```

```
[[1 4]
 [2 5]
 [3 6]]
```

iii)Built in Array functions

```
2]: #built in functions
m1 = [[1, 2, 3],[7,8,9]]
m2 = [4, 5, 6]
print(np.add(m1,m2))
print("----------------")
print(np.subtract(m1,m2))
print("----------------")
print(np.multiply(m1,m2))
print("----------------")
print(np.dot(m1,m2))
print("----------------")
mT=np.transpose(m1)
print(mT)
```

```
[[ 5  7  9]
 [11 13 15]]
----------------
[[-3 -3 -3]
 [ 3  3  3]]
----------------
[[ 4 10 18]
 [28 40 54]]
----------------
[ 32 122]
----------------
[[1 7]
 [2 8]
 [3 9]]
```

iv)
Read, write, append files

```
[15]: file1=open("C:\\Users\\RVUW249\\Desktop\\sample1.txt","x")
      file1.write("the content will be added in your file")
      file1.close()

[16]: file1=open("mya.txt","x") #x is for create
      file1.write("the content will be added in your file")
      file1.close()

[18]: file=open("mya.txt","r")
      res=file.read()
      print(res)

      the content will be added in your file

[20]: file=open("mya.txt","w")  #deletes old and overwrites
      file.write("nya")
      file.close()

[21]: file=open("mya.txt","r")
      res=file.read()
      print(res)

      nya

[24]: file=open("mya.txt","a") #appends so doesnt delete last items
      file.write("meow is also valid")
      file.close()

[23]: file=open("mya.txt","r")
      res=file.read()
      print(res)

      nyameow is also valid
```

v)Using with method

```
5]: with open("C:\\Users\\RVUW249\\Desktop\\sample1.txt","r") as file:
        print(file.read())

    the content will be added in your file

6]: with open("C:\\Users\\RVUW249\\Desktop\\sample1.txt","w") as file:
        print(file.write("new content"))

    11

7]: with open("C:\\Users\\RVUW249\\Desktop\\sample1.txt","w") as file:
        print(file.write("new content ++"))
    with open("C:\\Users\\RVUW249\\Desktop\\sample1.txt","r") as file:
        print(file.read())

    14
    new content ++
```

vi) Using csv files

```
[29]: import csv
      with open("C:\\Users\\RVUW249\\Downloads\\Untitled spreadsheet - Sheet1.csv") as file:
          reader = csv.reader(file)
          for row in reader:
              print(row)

      ['Name', 'Roll number']
      ['Adya', '1']
      ['Arnav', '2']
      ['Arya', '3']
      ['Abhijeet', '4']
      ['Akhil', '5']

[ ]:
```

i),ii) Using Try and Except Block

```
[1]: try:
         a=10
         b=0
         print(a/b)
     except:
         print("Error occured!")
```

```
Error occured!
```

```
[3]: try:
         num = int(input("Enter number: "))
         print(num)
     except ValueError:
         print("Invalid!")
```

```
Enter number:  hi
Invalid!
```

```
[ ]: try:
         a = int(input("Enter number: "))
         b = int(input("Enter number: "))
         print(a/b)
     except ValueError:
         print("Invalid!")
```

iii) Using multiple except blocks

```
Invalid!

[16]: try:
          a = int(input("Enter number: "))
          b = int(input("Enter number: "))
          print(a/b)
      except ValueError:
          print("Invalid!")
      except ZeroDivisionError:
          print("Cannot divide by zero!")
```

```
Enter number:  3/0
Invalid!
```

iv) Using else Block

```
7]: try:
        a = 10
        b = 0
        result = a/b
    except ZeroDivisionError:
        print("Division by zero not allowed!")
    else:
        print("Result: ", result)
```

```
Division by zero not allowed!
```

v) Using finally block

```
6]: try:
        f=open("data.txt","r")
        print(f.read)

    except FileNotFoundError:
        print("File not found!")
    finally:
        print("Execution completed!")
```

```
File not found!
Execution completed!
```

vi)Handling Multiple Exceptions in One Line

```
try:
    num = int("abc")
except (ValueError, TypeError):
    print("Error occured while converting value!")
```

```
Error occured while converting value!
```

vii) Raising an exception

```
: age = -5
  try:
      if age <0:
          raise ValueError("Age cannot be negative!")
  except ValueError as e:
      print("Error in age!")
```

```
Error in age!
```

viii) User defined exception

```
4]: class MyError(Exception):
        pass
    try:
        raise MyError("Custom Error occurred!")
    except MyError as e:
        print(e)
```

```
Custom Error occurred!
```

ix) Try-Except with File Handling

```
[5]: try:
         file = open("sample.txt", "r")
         content = file.read()
         print(content)
     except FileNotFoundError:
         print("File Does Not Exist!")
```

```
File Does Not Exist!
```

x) Catching Exception Object

```
: try:
      x=5/0
  except Exception as e:
      print("Error Message: ", e)
```

```
Error Message:  division by zero
```

Introduction to Pandas

i) Sample data csv files

```
[9]: #----------------------------------------PANDAS!!----------------------------------------
     import pandas as pd
     data = pd.Series([10,20,30,40])
     print(data)
```

```
0    10
1    20
2    30
3    40
dtype: int64
```

```
11]: data=pd.Series([10,20,30],index=["x","y","z"])
     print(data)
```

```
x    10
y    20
z    30
dtype: int64
```

```
12]: df=pd.DataFrame(data)
     print(df)
     print("\n")
```

```
    0
x  10
y  20
z  30
```

ii)Dictionary output printing

```
[6]: import pandas as pd
data = {
    "Name": ["Ram", "Vani"], "Age": [20,25]}

df = pd.DataFrame(data)
print(df)
print("\n")
print(df.Name)
print("---")

print(df.loc[0]) #0th row, particular col, accesses base on row
```

```
   Name  Age
0   Ram   20
1  Vani   25


0     Ram
1    Vani
Name: Name, dtype: str
---
Name    Ram
Age      20
Name: 0, dtype: object
```

iii) Opening a csv file

```
12]: df = pd.read_csv("C:\\Users\\RVUW249\\Desktop\\renamed.csv")
print(df)
```

```
       Year Industry_aggregation_NZSIOC Industry_code_NZSIOC  \
0      2024                     Level 1                99999
1      2024                     Level 1                99999
2      2024                     Level 1                99999
3      2024                     Level 1                99999
4      2024                     Level 1                99999
...     ...                         ...                  ...
55615  2013                     Level 3                 ZZ11
55616  2013                     Level 3                 ZZ11
55617  2013                     Level 3                 ZZ11
55618  2013                     Level 3                 ZZ11
55619  2013                     Level 3                 ZZ11

             Industry_name_NZSIOC               Units Variable_code  \
0                 All industries  Dollars (millions)           H01
1                 All industries  Dollars (millions)           H04
2                 All industries  Dollars (millions)           H05
3                 All industries  Dollars (millions)           H07
4                 All industries  Dollars (millions)           H08
...                          ...                 ...           ...
55615  Food product manufacturing          Percentage           H37
55616  Food product manufacturing          Percentage           H38
55617  Food product manufacturing          Percentage           H39
55618  Food product manufacturing          Percentage           H40
55619  Food product manufacturing          Percentage           H41

                                       Variable_name      Variable_category  \
0                                       Total income  Financial performance
1      Sales, government funding, grants and subsidies  Financial performance
2                    Interest, dividends and donations  Financial performance
3                                 Non-operating income  Financial performance
4                                    Total expenditure  Financial performance
...                                                ...                   ...
55615                                       Quick ratio       Financial ratios
```

iv) A specific column to print

```
[14]: print(df.Value)
      print(df.Variable_code)
```

```
0        979594
1        838626
2        112188
3         28781
4        856960
          ...
55615        52
55616        40
55617        12
55618         5
55619        46
Name: Value, Length: 55620, dtype: str
0        H01
1        H04
2        H05
3        H07
4        H08
        ...
55615    H37
55616    H38
55617    H39
55618    H40
55619    H41
Name: Variable_code, Length: 55620, dtype: str
```

v) Table to csv file

```
[16]: #table to csv
      data = {
          "Name": ["Ram", "Vani"], "Age": [20,25]}
      df = pd.DataFrame(data)
      df.to_csv("firstt.csv",index=False)
      print("created")
```

```
created
```

vi)To print only the first 5 rows

```
43]: df = pd.read_csv("C:\\Users\\RVUW249\\Desktop\\renamed.csv")
     df.head()
```

43]:

	Year	Industry_aggregation_NZSIOC	Industry_code_NZSIOC	Industry_name_NZSIOC	Units	Variable_code	Variable_name	Variable_category	Value	Industry_cod
0	2024	Level 1	99999	All industries	Dollars (millions)	H01	Total income	Financial performance	979594	ANZSIC06 (excluding c
1	2024	Level 1	99999	All industries	Dollars (millions)	H04	Sales, government funding, grants and subsidies	Financial performance	838626	ANZSIC06 (excluding c
2	2024	Level 1	99999	All industries	Dollars (millions)	H05	Interest, dividends and donations	Financial performance	112188	ANZSIC06 (excluding c
3	2024	Level 1	99999	All industries	Dollars (millions)	H07	Non-operating income	Financial performance	28781	ANZSIC06 (excluding c
4	2024	Level 1	99999	All industries	Dollars (millions)	H08	Total expenditure	Financial performance	856960	ANZSIC06 (excluding c

vii) shape, number of dimensions

```
df = pd.read_csv("C:\\Users\\RVUW249\\Desktop\\renamed.csv")
df.shape
```

```
(55620, 10)
```

```
df = pd.read_csv("C:\\Users\\RVUW249\\Desktop\\renamed.csv")
df.ndim
```

```
2
```

viii)Total number of rows in the csv file

```
df = pd.read_csv("C:\\Users\\RVUW249\\Desktop\\renamed.csv")
len(df)
```

```
55620
```

ix)Last 5 rows of the csv file

```
df = pd.read_csv("C:\\Users\\RVUW249\\Desktop\\renamed.csv")
df.tail()
```

	Year	Industry_aggregation_NZSIOC	Industry_code_NZSIOC	Industry_name_NZSIOC	Units	Variable_code	Variable_name	Variable_category	Value	Industr
55615	2013	Level 3	ZZ11	Food product manufacturing	Percentage	H37	Quick ratio	Financial ratios	52	ANZS C112, C
55616	2013	Level 3	ZZ11	Food product manufacturing	Percentage	H38	Margin on sales of goods for resale	Financial ratios	40	ANZS C112, C
55617	2013	Level 3	ZZ11	Food product manufacturing	Percentage	H39	Return on equity	Financial ratios	12	ANZS C112, C
55618	2013	Level 3	ZZ11	Food product manufacturing	Percentage	H40	Return on total assets	Financial ratios	5	ANZS C112, C
55619	2013	Level 3	ZZ11	Food product manufacturing	Percentage	H41	Liabilities structure	Financial ratios	46	ANZS C112, C

x) Displaying index of your file

```
df.index
```

```
RangeIndex(start=0, stop=55620, step=1)
```

xi) Displaying the column names

```
df = pd.read_csv("C:\\Users\\RVUW249\\Desktop\\renamed.csv")
df.columns
```

```
Index(['Year', 'Industry_aggregation_NZSIOC', 'Industry_code_NZSIOC',
       'Industry_name_NZSIOC', 'Units', 'Variable_code', 'Variable_name',
       'Variable_category', 'Value', 'Industry_code_ANZSIC06'],
      dtype='str')
```

xii) To fill the empty cell which has the value 0

[59]:
```
df = pd.read_csv("C:\\Users\\RVUW249\\Desktop\\renamed.csv")
df.fillna(0)
```

[59]:

	Year	Industry_aggregation_NZSIOC	Industry_code_NZSIOC	Industry_name_NZSIOC	Units	Variable_code	Variable_name	Variable_category	Value	Indust
0	2024	Level 1	99999	All industries	Dollars (millions)	H01	Total income	Financial performance	979594	AN (excl
1	2024	Level 1	99999	All industries	Dollars (millions)	H04	Sales, government funding, grants and subsidies	Financial performance	838626	AN (excl
2	2024	Level 1	99999	All industries	Dollars (millions)	H05	Interest, dividends and donations	Financial performance	112188	AN (excl
3	2024	Level 1	99999	All industries	Dollars (millions)	H07	Non-operating income	Financial performance	28781	AN (excl
4	2024	Level 1	99999	All industries	Dollars (millions)	H08	Total expenditure	Financial performance	856960	AN (excl
...	...	...	...	...	...	...	...	...	...	
55615	2013	Level 3	ZZ11	Food product manufacturing	Percentage	H37	Quick ratio	Financial ratios	52	AN: C112,
55616	2013	Level 3	ZZ11	Food product manufacturing	Percentage	H38	Margin on sales of goods for resale	Financial ratios	40	AN: C112,
55617	2013	Level 3	ZZ11	Food product manufacturing	Percentage	H39	Return on equity	Financial ratios	12	AN: C112,
...	...	...	...	...	...	...	...	...	...	
55615	2013	Level 3	ZZ11	Food product manufacturing	Percentage	H37	Quick ratio	Financial ratios	52	AN: C112,
55616	2013	Level 3	ZZ11	Food product manufacturing	Percentage	H38	Margin on sales of goods for resale	Financial ratios	40	AN: C112,
55617	2013	Level 3	ZZ11	Food product manufacturing	Percentage	H39	Return on equity	Financial ratios	12	AN: C112,
55618	2013	Level 3	ZZ11	Food product manufacturing	Percentage	H40	Return on total assets	Financial ratios	5	AN: C112,
55619	2013	Level 3	ZZ11	Food product manufacturing	Percentage	H41	Liabilities structure	Financial ratios	46	AN: C112,

55620 rows × 10 columns

xiii)To remove the missing data

[61]:
```
df.dropna()
```

[61]:

	Year	Industry_aggregation_NZSIOC	Industry_code_NZSIOC	Industry_name_NZSIOC	Units	Variable_code	Variable_name	Variable_category	Value	Indust
0	2024	Level 1	99999	All industries	Dollars (millions)	H01	Total income	Financial performance	979594	AN (excl
1	2024	Level 1	99999	All industries	Dollars (millions)	H04	Sales, government funding, grants and subsidies	Financial performance	838626	AN (excl
2	2024	Level 1	99999	All industries	Dollars (millions)	H05	Interest, dividends and donations	Financial performance	112188	AN (excl
3	2024	Level 1	99999	All industries	Dollars (millions)	H07	Non-operating income	Financial performance	28781	AN (excl
4	2024	Level 1	99999	All industries	Dollars (millions)	H08	Total expenditure	Financial performance	856960	AN (excl
...	...	...	...	...	...	...	...	...	...	
55615	2013	Level 3	ZZ11	Food product manufacturing	Percentage	H37	Quick ratio	Financial ratios	52	AN: C112,
55616	2013	Level 3	ZZ11	Food product manufacturing	Percentage	H38	Margin on sales of goods for resale	Financial ratios	40	AN: C112,
55617	2013	Level 3	ZZ11	Food product manufacturing	Percentage	H39	Return on equity	Financial ratios	12	AN: C112,
55618	2013	Level 3	ZZ11	Food product manufacturing	Percentage	H40	Return on total assets	Financial ratios	5	AN: C112,

...	...	...	...	...	...	...	...	...	...	
55615	2013	Level 3	ZZ11	Food product manufacturing	Percentage	H37	Quick ratio	Financial ratios	52	C
55616	2013	Level 3	ZZ11	Food product manufacturing	Percentage	H38	Margin on sales of goods for resale	Financial ratios	40	C
55617	2013	Level 3	ZZ11	Food product manufacturing	Percentage	H39	Return on equity	Financial ratios	12	C
55618	2013	Level 3	ZZ11	Food product manufacturing	Percentage	H40	Return on total assets	Financial ratios	5	C
55619	2013	Level 3	ZZ11	Food product manufacturing	Percentage	H41	Liabilities structure	Financial ratios	46	C

55620 rows × 10 columns

xiv) To show the empty rows

```
[62]: df.isnull()
```

[62]:

	Year	Industry_aggregation_NZSIOC	Industry_code_NZSIOC	Industry_name_NZSIOC	Units	Variable_code	Variable_name	Variable_category	Value	Industry_cod
0	False	False	False	False	False	False	False	False	False	
1	False	False	False	False	False	False	False	False	False	
2	False	False	False	False	False	False	False	False	False	
3	False	False	False	False	False	False	False	False	False	
4	False	False	False	False	False	False	False	False	False	
...	...	...	...	...	...	...	...	...	...	
55615	False	False	False	False	False	False	False	False	False	
55616	False	False	False	False	False	False	False	False	False	
55617	False	False	False	False	False	False	False	False	False	
55618	False	False	False	False	False	False	False	False	False	
55619	False	False	False	False	False	False	False	False	False	

55620 rows × 10 columns

i) Using os commands in numpy

```
[20]: #shutil means shell util
      import os
      os.getcwd()
```

```
[20]: 'C:\\Users\\RVUW249'
```

ii)Printing all directory in list

```
[21]: os.listdir()
```

```
[21]: ['.bash_history',
       '.copilot',
       '.gitconfig',
       '.gitignore',
       '.gitinore',
       '.idlerc',
       '.ipynb_checkpoints',
       '.ipython',
       '.jupyter',
       '.lesshst',
       '.python_history',
       '.viminfo',
       '.vscode',
       'AppData',
       'Application Data',
       'ccomplexrepo',
       'complexrpo',
       'Contacts',
       'Cookies',
       'Desktop',
       'do-not-click.ipynb',
       'Documents',
       'Downloads',
       'f.r.i.e.s',
       'Favorites',
       'firstt.csv',
       'ivc',
       'Links',
       'Local Settings',
       'matcha-case.ipynb',
       'Music',
       'My Documents',
       'my-portfolio-site',
       'mya.txt',
```

iii) To check of a file exists

```
[25]: os.path.exists("renamed.csv")
```

```
[25]: False
```

iv) To make a new directory

```
[26]: os.mkdir("directoryyy")
```

```
[32]: os.mkdir("itsmemario")
```

v) To rename a file

```
[28]: os.rename("firstt.csv","firsth.csv")
```

vi) To copy a file using shutil

```
[29]: import shutil
      shutil.copy("firsth.csv","directoryyy")

[29]: 'directoryyy\\firsth.csv'
```

vii) To move a file using shutil

```
3]: import shutil
    shutil.move("firsth.csv", "itsmemario")

3]: 'itsmemario\\firsth.csv'
```

viii)To remove a directory

```
34]: shutil.rmtree("directoryyy")
```

ix)Gives current working directory

```
63]: os.getcwd

63]: <function nt.getcwd()>
```

i) To create a dataframe, add value and a calculated additional column

```
: import pandas as pd


data = {
    'Product': ['Laptop', 'Smartphone', 'Tablet', 'Headphones'],
    'Price': [1200, 800, 450, 150]
}
df = pd.DataFrame(data)
# 3. Add a new column
df['Stock'] = [15, 30, 10, 50]
# 4. Add a calculated column (e.g., 'Total Value')
df['Total Value'] = df['Price'] * df['Stock']
print("--- Updated DataFrame ---")
print(df)
```

```
--- Updated DataFrame ---
      Product  Price  Stock  Total Value
0      Laptop   1200     15        18000
1  Smartphone    800     30        24000
2      Tablet    450     10         4500
3  Headphones    150     50         7500
```

ii) Printing top 6 rows information and using df.info

```
[3]: import pandas as pd

     df = pd.read_csv(r'C:\Users\RVUW249\Desktop\renamed.csv')
     print(df.head(6))
```

```
   Year Industry_aggregation_NZSIOC Industry_code_NZSIOC Industry_name_NZSIOC  \
0  2024                     Level 1                99999       All industries
1  2024                     Level 1                99999       All industries
2  2024                     Level 1                99999       All industries
3  2024                     Level 1                99999       All industries
4  2024                     Level 1                99999       All industries
5  2024                     Level 1                99999       All industries

               Units Variable_code  \
0  Dollars (millions)           H01
1  Dollars (millions)           H04
2  Dollars (millions)           H05
3  Dollars (millions)           H07
4  Dollars (millions)           H08
5  Dollars (millions)           H09

                                  Variable_name      Variable_category  \
0                                  Total income  Financial performance
1  Sales, government funding, grants and subsidies  Financial performance
2               Interest, dividends and donations  Financial performance
3                            Non-operating income  Financial performance
4                               Total expenditure  Financial performance
5                          Interest and donations  Financial performance

    Value                            Industry_code_ANZSIC06
0  979594  ANZSIC06 divisions A-S (excluding classes K633...
1  838626  ANZSIC06 divisions A-S (excluding classes K633...
2  112188  ANZSIC06 divisions A-S (excluding classes K633...
3   28781  ANZSIC06 divisions A-S (excluding classes K633...
4  856960  ANZSIC06 divisions A-S (excluding classes K633...
5   71493  ANZSIC06 divisions A-S (excluding classes K633...
```

```
[4]: df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 55620 entries, 0 to 55619
Data columns (total 10 columns):
 #   Column                       Non-Null Count  Dtype
---  ------                       --------------  -----
 0   Year                         55620 non-null  int64
 1   Industry_aggregation_NZSIOC  55620 non-null  str
 2   Industry_code_NZSIOC         55620 non-null  str
 3   Industry_name_NZSIOC         55620 non-null  str
 4   Units                        55620 non-null  str
 5   Variable_code                55620 non-null  str
 6   Variable_name                55620 non-null  str
 7   Variable_category            55620 non-null  str
 8   Value                        55620 non-null  str
 9   Industry_code_ANZSIC06       55620 non-null  str
dtypes: int64(1), str(9)
memory usage: 4.2 MB
```

```
[ ]:
```

iii) Add a new column which has new values of top 6 rows

```
[13]: import pandas as pd
df = pd.read_csv("C:\\Users\\RVUW249\\Desktop\\renamed.csv")

df['Value'] = df['Value'].astype(str).str.replace(',', '', regex=False).replace('C', '0')

df['mean'] = df['Industry_code_NZSIOC'].astype(str) + df['Value']

print(df[['Industry_code_NZSIOC', 'Value', 'mean']].head(6))
```

```
  Industry_code_NZSIOC   Value         mean
0                99999  979594  99999979594
1                99999  838626  99999838626
2                99999  112188  99999112188
3                99999   28781   9999928781
4                99999  856960  99999856960
5                99999   71493   9999971493
```

iv) Change the empty cell value with mean

[19]:
```
import pandas as pd

df = pd.read_csv("C:\\Users\\RVUW249\\Downloads\\renamed - renamed.csv.csv")

df['Value'] = pd.to_numeric(df['Value'], errors='coerce')


mean_val = df['Value'].mean()

df['Value'] = df['Value'].fillna(mean_val)

print(df)
```

```
       Year Industry_aggregation_NZSIOC Industry_code_NZSIOC  \
0      2024                    Level 1               99999
1      2024                    Level 1               99999
2      2024                    Level 1                 NaN
3      2024                    Level 1               99999
4      2024                    Level 1               99999
...     ...                        ...                 ...
55615  2013                    Level 3                ZZ11
55616  2013                    Level 3                ZZ11
55617  2013                    Level 3                ZZ11
55618  2013                    Level 3                ZZ11
55619  2013                    Level 3                ZZ11

             Industry_name_NZSIOC               Units Variable_code  \
0                 All industries  Dollars (millions)           H01
1                 All industries  Dollars (millions)           H04
2                 All industries  Dollars (millions)           H05
3                 All industries  Dollars (millions)           H07
4                 All industries  Dollars (millions)           H08
...                          ...                 ...           ...
55615  Food product manufacturing          Percentage           H37
55616  Food product manufacturing          Percentage           H38
55617  Food product manufacturing          Percentage           H39
```

iv) Displaying mean, median, mode, var, std, min, max

[21]:
```
print(df.Value.mean())
print(df.Value.median())
print(df.Value.mode())
print(df.Value.min())
print(df.Value.max())
print(df.Value.var())
print(df.Value.std())
```

```
19855.160471553554
652.0
0    19855.160472
Name: Value, dtype: float64
-152300.0
12475400.0
25283966348.299595
159009.32786569346
```

[ ]:

Lab 9th

i) To calculate descriptive statistics, quantiles, skewness, and kurtosis for understanding the data distribution

Input

```
[2]: #-----------------------LAB 9
import pandas as pd
df = pd.read_csv("C:\\Users\\RVUW249\\Downloads\\renamed - renamed.csv.csv")

df['Value'] = pd.to_numeric(df['Value'].astype(str).str.replace(',', ''), errors='coerce')
df = df.dropna(subset=['Value'])

print("--- Statistical Summary ---")
print(df['Value'].describe())

print("\n--- Quantiles (25%, 50%, 75%) ---")
print(df['Value'].quantile([0.25, 0.5, 0.75]))


skewness = df['Value'].skew()
kurtosis = df['Value'].kurt()

print(f"\nSkewness: {skewness}")
print(f"Kurtosis: {kurtosis}")
```

Output:

```
--- Statistical Summary ---
count    5.321200e+04
mean     3.310768e+04
std      2.502369e+05
min     -1.523000e+05
25%      5.800000e+01
50%      5.290000e+02
75%      4.162250e+03
max      1.247540e+07
Name: Value, dtype: float64

--- Quantiles (25%, 50%, 75%) ---
0.25      58.00
0.50     529.00
0.75    4162.25
Name: Value, dtype: float64

Skewness: 23.698959700036497
Kurtosis: 785.2973754574712
```

Lab 10: To convert categorical text data into binary numerical columns using One-Hot Encoding for machine learning compatibility

Input:

```
4]: #------LAB 10
import pandas as pd
from sklearn.preprocessing import OneHotEncoder

# 1. Load the dataset
df = pd.read_csv("C:\\Users\\RVUW249\\Downloads\\renamed - renamed.csv.csv")


enc = OneHotEncoder()

enc_data = enc.fit_transform(df[['Variable_category']])

column_names = enc.get_feature_names_out(['Variable_category'])

enc_df = pd.DataFrame(enc_data.toarray(), columns=column_names)

df_final = df.join(enc_df)

print("--- List of Categories Identified ---")
print(enc.categories_)

print("\n--- Encoded DataFrame (First 5 rows) ---")
print(enc_df.head())

print("\n--- Final Combined DataFrame ---")
print(df_final.head())
```

Output:

```
--- List of Categories Identified ---
[array(['Financial performance', 'Financial position', 'Financial ratios'],
      dtype=object)]

--- Encoded DataFrame (First 5 rows) ---
   Variable_category_Financial performance  \
0                                       1.0
1                                       1.0
2                                       1.0
3                                       1.0
4                                       1.0

   Variable_category_Financial position  Variable_category_Financial ratios
0                                   0.0                                 0.0
1                                   0.0                                 0.0
2                                   0.0                                 0.0
3                                   0.0                                 0.0
4                                   0.0                                 0.0

--- Final Combined DataFrame ---
   Year Industry_aggregation_NZSIOC Industry_code_NZSIOC Industry_name_NZSIOC  \
0  2024                     Level 1                99999       All industries
1  2024                     Level 1                99999       All industries
2  2024                     Level 1                  NaN       All industries
3  2024                     Level 1                99999       All industries
4  2024                     Level 1                99999       All industries

               Units Variable_code  \
0  Dollars (millions)           H01
1  Dollars (millions)           H04
2  Dollars (millions)           H05
3  Dollars (millions)           H07
4  Dollars (millions)           H08

                                     Variable_name      Variable_category  \
0                                     Total income  Financial performance
1  Sales, government funding, grants and subsidies  Financial performance
2                Interest, dividends and donations  Financial performance
3                            Non-operating income  Financial performance
4                               Total expenditure  Financial performance

    Value                              Industry_code_ANZSIC06  \
0  979594  ANZSIC06 divisions A-S (excluding classes K633...
1  838626  ANZSIC06 divisions A-S (excluding classes K633...
2     NaN  ANZSIC06 divisions A-S (excluding classes K633...
3   28781  ANZSIC06 divisions A-S (excluding classes K633...
4  856960  ANZSIC06 divisions A-S (excluding classes K633...

   Variable_category_Financial performance  \
0                                       1.0
```

```
                    Units Variable_code  \
0  Dollars (millions)           H01
1  Dollars (millions)           H04
2  Dollars (millions)           H05
3  Dollars (millions)           H07
4  Dollars (millions)           H08

                                     Variable_name      Variable_category  \
0                                     Total income  Financial performance
1  Sales, government funding, grants and subsidies  Financial performance
2                Interest, dividends and donations  Financial performance
3                             Non-operating income  Financial performance
4                                Total expenditure  Financial performance

    Value                            Industry_code_ANZSIC06  \
0  979594  ANZSIC06 divisions A-S (excluding classes K633...
1  838626  ANZSIC06 divisions A-S (excluding classes K633...
2     NaN  ANZSIC06 divisions A-S (excluding classes K633...
3   28781  ANZSIC06 divisions A-S (excluding classes K633...
4  856960  ANZSIC06 divisions A-S (excluding classes K633...

   Variable_category_Financial performance  \
0                                      1.0
1                                      1.0
2                                      1.0
3                                      1.0
4                                      1.0

   Variable_category_Financial position  Variable_category_Financial ratios
0                                   0.0                                 0.0
1                                   0.0                                 0.0
2                                   0.0                                 0.0
3                                   0.0                                 0.0
4                                   0.0                                 0.0
```

Lab 11: To implement data scaling and normalization techniques like StandardScaler, MinMaxScaler, and Normalizer to standardize numerical ranges

Input:

```python
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler, MinMaxScaler, Normalizer

df = pd.read_csv("C:\\Users\\RVUW249\\Downloads\\renamed - renamed.csv.csv")

df['Value'] = pd.to_numeric(df['Value'].astype(str).str.replace(',', ''), errors='coerce')
df = df.dropna(subset=['Value'])
data_values = df[['Value']]

scaler_std = StandardScaler()
scaled_std = scaler_std.fit_transform(data_values)
print(scaled_std)

scaler_mm = MinMaxScaler()
scaled_mm = scaler_mm.fit_transform(data_values)
print(scaled_mm)

scaler_norm = Normalizer()
normalized_data = scaler_norm.fit_transform(data_values)
print(normalized_data)
```

Output:

```
[[ 3.7823966 ]
 [ 3.21905313]
 [-0.01729048]
 ...
 [-0.13225862]
 [-0.13228659]
 [-0.13212274]]
[[0.0896358 ]
 [0.07847241]
 [0.01433998]
 ...
 [0.01206174]
 [0.01206118]
 [0.01206443]]
[[1.]
 [1.]
 [1.]
 ...
 [1.]
 [1.]
 [1.]]
```

Lab 12: To make boxplots using Matplotlib and Seaborn and identify outliers in salary data using the Interquartile Range (IQR) method

Input:

```python
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Load dataset (Lab 12)
df = pd.read_csv("nyancat.csv")

print("Dataset Head:")
print(df.head())


# Matplotlib Boxplot - Salary
print("\nBoxplot for Salary")

# notch=True adds the confidence interval around the median
# vert=False makes the plot horizontal
plt.boxplot(df["Salary"], notch=True, vert=False, showmeans=True)
plt.xlabel("Salary Amount")
plt.title("Boxplot - Salary Distribution")
plt.show()


# Matplotlib Boxplot - Height
# -------------------------------
print("\nBoxplot for Height cm")

plt.boxplot(df["Height cm"])
plt.ylabel("Height (cm)")
plt.title("Boxplot - Height")
plt.show()
```

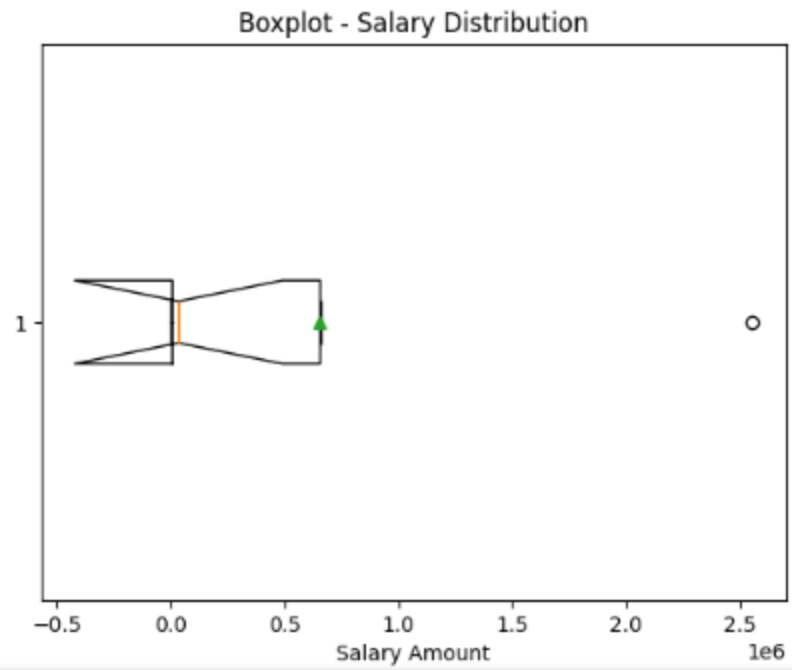
```python
# Seaborn Boxplot - Salary
# ------------------------------
print("\nSeaborn Boxplot for Salary")

# Seaborn handles labels and styling automatically
sns.boxplot(x=df["Salary"])
plt.title("Seaborn Boxplot - Salary")
plt.show()

print("\nSeaborn Boxplot - Salary vs Marks")

# Comparing a numerical value across categories or ranges
sns.boxplot(x=df["Salary"], y=df["Marks"].astype(str)) # converting Marks to string to treat as category
plt.title("Salary vs Marks Distribution")
plt.show()


# IQR Method for Salary
# ------------------------------
print("\nIQR Method Calculation for Salary")

Q1 = df["Salary"].quantile(0.25)
Q3 = df["Salary"].quantile(0.75)

print("Q1 (25th Percentile):", Q1)
print("Q3 (75th Percentile):", Q3)

IQR = Q3 - Q1
print("IQR (Interquartile Range):", IQR)

# Calculating the boundaries for outliers
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR
```

```python
# Calculating the boundaries for outliers
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

print("Lower Bound (Whisker):", lower)
print("Upper Bound (Whisker):", upper)

# Filtering the DataFrame to find rows outside the bounds
outliers = df[(df["Salary"] < lower) | (df["Salary"] > upper)]

print("\nOutliers detected in Salary:")
if outliers.empty:
    print("No outliers found.")
else:
    print(outliers)
```

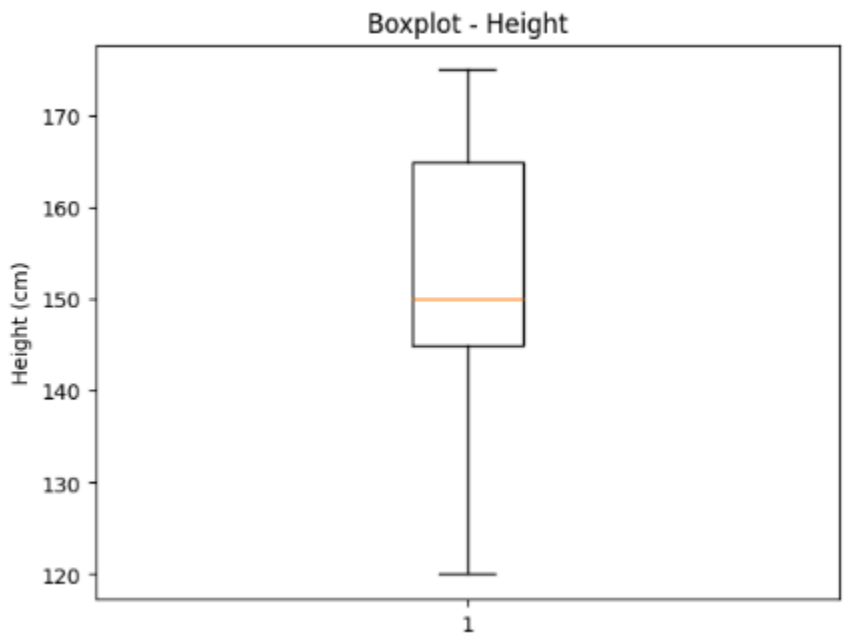
Output:

```
Dataset Head:
       Name  Roll number  Marks  Height cm   Salary
0      Adya            1     30        175  2553534
1     Arnav            2     40        150    34646
2      Arya            3     50        120   656454
3  Abhijeet            4     60        145      453
4     Akhil            5     70        165     7000
```

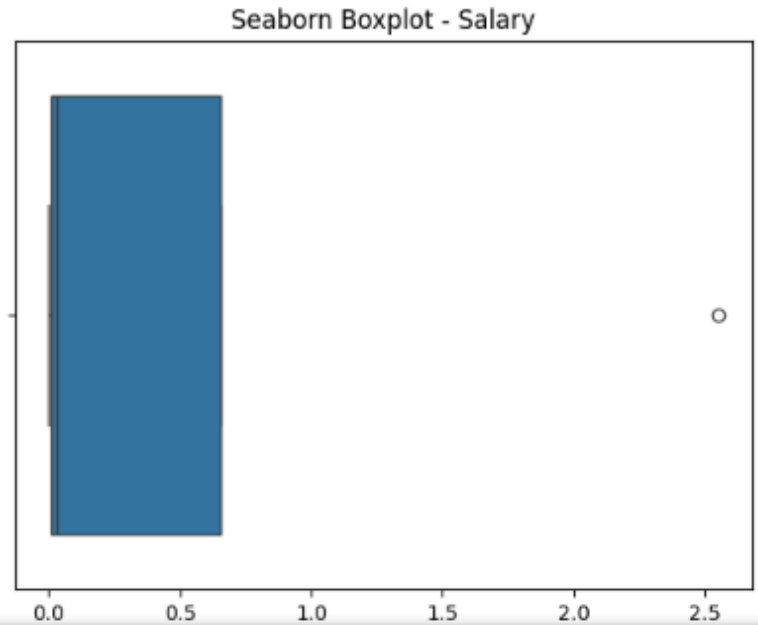
```
Boxplot for Salary
```



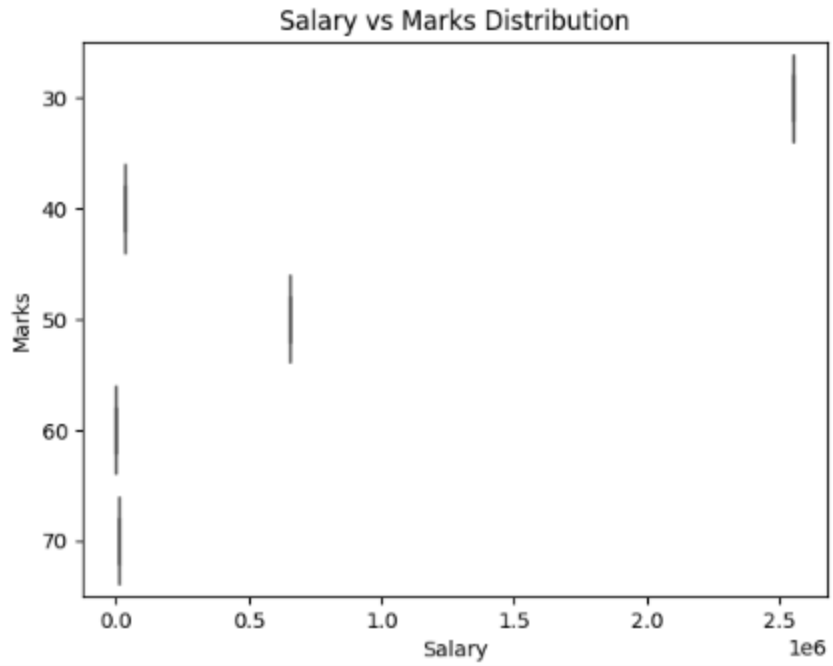

```
Boxplot for Height cm
```



```
Seaborn Boxplot for Salary
```



```
Seaborn Boxplot - Salary vs Marks
```




```
IQR Method Calculation for Salary
Q1 (25th Percentile): 7000.0
Q3 (75th Percentile): 656454.0
IQR (Interquartile Range): 649454.0
Lower Bound (Whisker): -967181.0
Upper Bound (Whisker): 1630635.0

Outliers detected in Salary:
   Name  Roll number  Marks  Height cm   Salary
0  Adya            1     30        175  2553534
```

Lab 13: To perform data cleaning and transformation by splitting, merging, and joining dataframes and calculating group-based statistics like average salary

Input:

```python
#group by function for distinct units
import pandas as pd

# Load the data from nyancat.csv
df = pd.read_csv("nyancat.csv")

# --- 1. DataFrames Creation ---
# Extracting basic info and adding a 'Dept' column for the example
emp_info = df[['Roll number', 'Name']].copy()
emp_info['Dept'] = ['HR', 'Eng', 'Eng', 'HR', 'Mkt']

# Extracting salary info
emp_sal = df[['Roll number', 'Salary']].copy()

# Generating temporal sales data
sales_jan = pd.DataFrame({
    'Date': pd.date_range(start='2026-01-01', periods=len(df)),
    'Reg': ['N'] * len(df),
    'Sale': [200, 250, 300, 150, 400]
})

sales_feb = pd.DataFrame({
    'Date': pd.date_range(start='2026-02-01', periods=len(df)),
    'Reg': ['S'] * len(df),
    'Sale': [210, 240, 310, 160, 390]
})

print("Employee Info (from nyancat):")
print(emp_info)

print("\nSalary Data (from nyancat):")
print(emp_sal)
```

```python
# --- 2. Merging ---
# Merging on 'Roll number'
merge_data = pd.merge(emp_info, emp_sal, on='Roll number')

print("\nMerged Data:")
print(merge_data)


# --- 3. GroupBy ---
# Calculating Average Salary by Dept
avg = emp_info.merge(emp_sal, on='Roll number').groupby('Dept')['Salary'].mean().reset_index()
avg.columns = ['dept', 'avgsal']

print("\nAverage Salary by Dept:")
print(avg)


# --- 4. Joining ---
# Setting index to 'Roll number' for joining
emp1 = emp_info.set_index('Roll number')
sal1 = emp_sal.set_index('Roll number')
join_data = emp1.join(sal1, how='inner')

print("\nJoined Data (using index):")
print(join_data)


# --- 5. Concatenation ---
# Concatenating Sales Data horizontally
final = pd.concat([sales_jan, sales_feb], axis=1)

print("\nConcatenated Sales Data:")
print(final)
```

Output:

```
Employee Info (from nyancat):
   Roll number      Name Dept
0            1      Adya   HR
1            2     Arnav  Eng
2            3      Arya  Eng
3            4  Abhijeet   HR
4            5     Akhil  Mkt

Salary Data (from nyancat):
   Roll number   Salary
0            1  2553534
1            2    34646
2            3   656454
3            4      453
4            5     7000

Merged Data:
   Roll number      Name Dept   Salary
0            1      Adya   HR  2553534
1            2     Arnav  Eng    34646
2            3      Arya  Eng   656454
3            4  Abhijeet   HR      453
4            5     Akhil  Mkt     7000

Average Salary by Dept:
  dept     avgsal
0  Eng   345550.0
1   HR  1276993.5
2  Mkt     7000.0


  Joined Data (using index):
                   Name Dept   Salary
  Roll number
  1                 Adya   HR  2553534
  2                Arnav  Eng    34646
  3                 Arya  Eng   656454
  4             Abhijeet   HR      453
  5                Akhil  Mkt     7000

  Concatenated Sales Data:
          Date Reg  Sale       Date Reg  Sale
  0 2026-01-01   N   200 2026-02-01   S   210
  1 2026-01-02   N   250 2026-02-02   S   240
  2 2026-01-03   N   300 2026-02-03   S   310
  3 2026-01-04   N   150 2026-02-04   S   160
  4 2026-01-05   N   400 2026-02-05   S   390
```

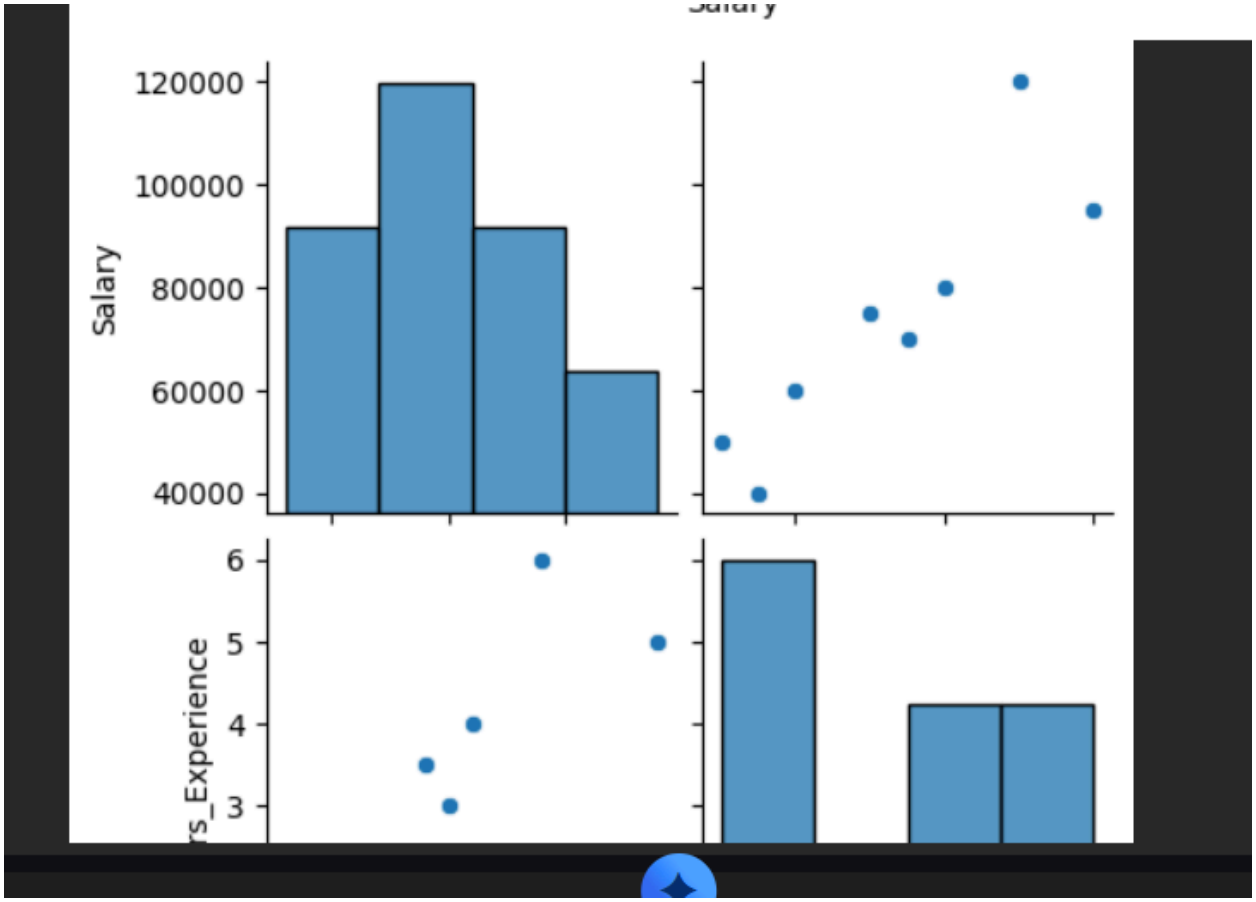
Labs 14-18
Plotting graphs with seaborn

```python
# lab 14 to 18
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

df = pd.read_csv("/content/12laptops - 12laptops.csv")
data = {
'Salary': [50000, 60000, 75000, 80000, 120000, 95000, 40000, 70000],
'Years_Experience': [1, 2, 3, 4, 5, 6, 1.5, 3.5],
'Experience_Level': ['Junior', 'Junior', 'Mid', 'Mid', 'Senior', 'Senior', 'Junior', 'Mid']
}
df = pd.DataFrame(data)

# 1) Salary Distribution
sns.histplot(df['Salary'], kde=True)
plt.title("Salary Distribution")
plt.show()

# 2) Pair Plot (Salary vs Years_Experience)
sns.pairplot(df[['Salary', 'Years_Experience']])
plt.show()

# 3) Correlation Heatmap
sns.heatmap(df[['Salary', 'Years_Experience']].corr(), annot=True, cmap='coolwarm')
plt.show()
```

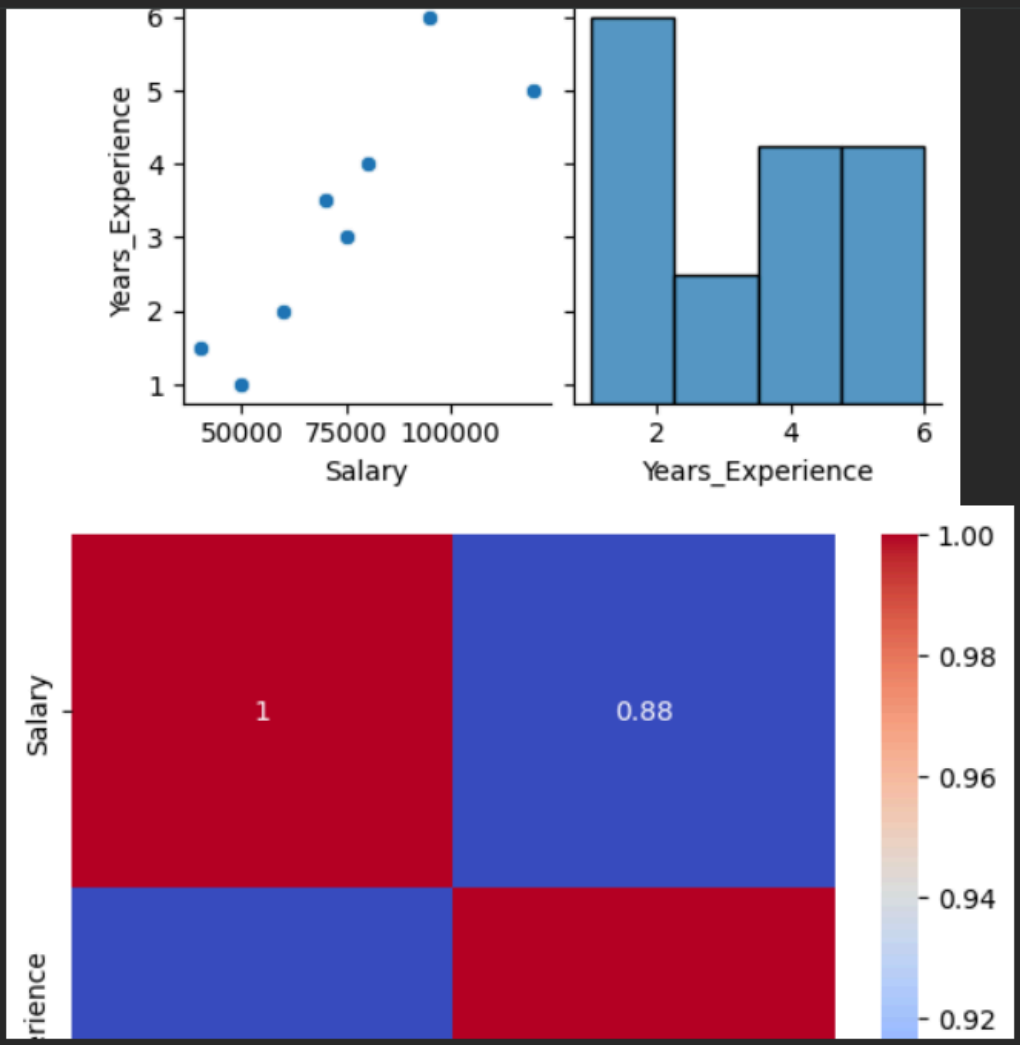
```python
# 3) Correlation Heatmap
sns.heatmap(df[['Salary', 'Years_Experience']].corr(), annot=True, cmap='coolwarm')
plt.show()


# 4) Line Plot: Salary Trend by Years of Experience
sns.lineplot(x='Years_Experience', y='Salary', data=df, marker='o')
         # 'o' - circle  's' (square), '^' (triangle), '*' (star) (don't type)
plt.title("Salary Trend by Experience")
plt.show()

# 5) Scatter Plot: Salary vs Years of Experience
sns.scatterplot(x='Years_Experience', y='Salary', data=df, s=100)
                                                        # s = size of each scatter point.
plt.title("Salary vs Experience")
plt.show()

# 6) Bar Plot: Experience Level vs Salary
sns.barplot(x='Experience_Level', y='Salary', data=df)
plt.title("Salary by Experience Level")
plt.show()
```

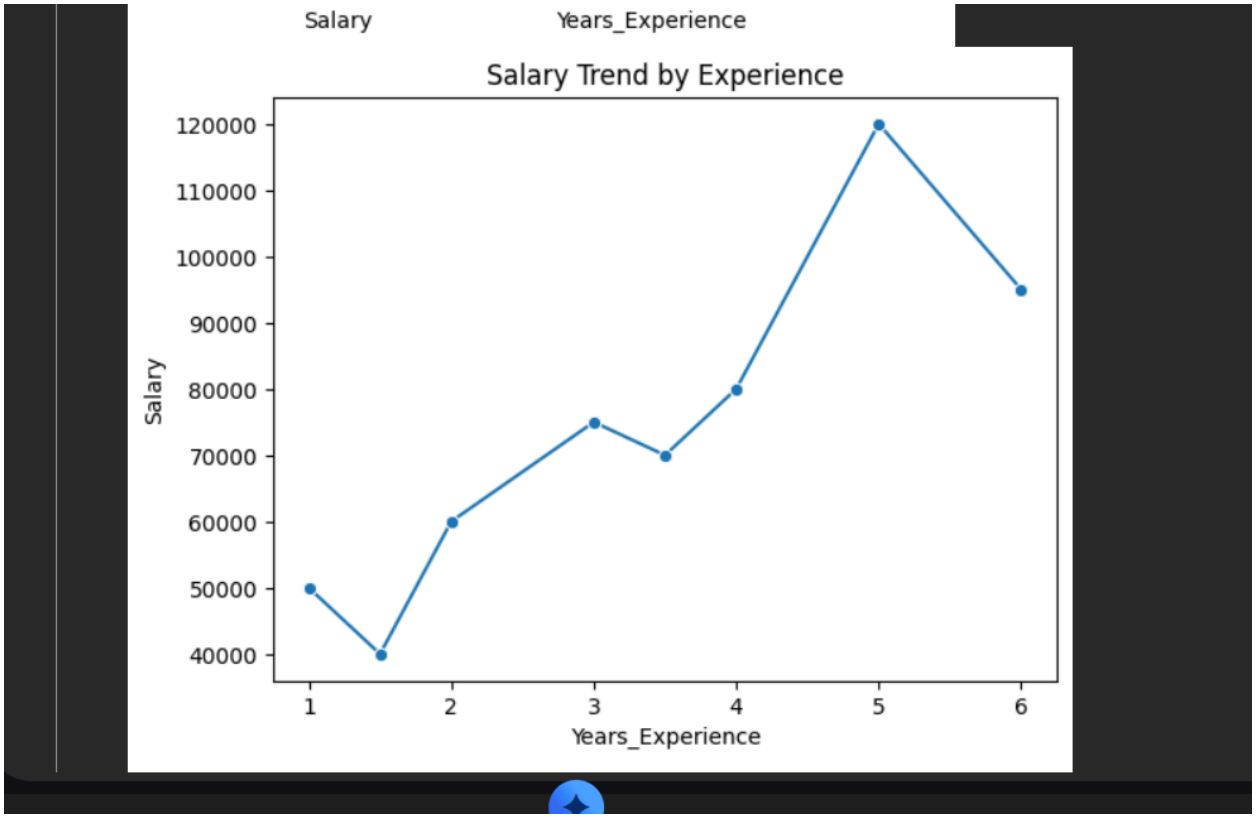
Output:

Salary Distribution
Count
3.0
2.5
2.0
1.5
1.0
0.5
0.0
40000
50000
60000
70000
80000
90000
100000
110000
120000
Salary
120000


Salary
120000
100000
80000
60000
40000
6
5
4
3
rs_Experience

```
plt.show()
```


Years_Experience
Salary
Years_Experience
Salary
1
0.88
1.00
0.98
0.96
0.94
0.92


Salary
Years_Experience
Salary Trend by Experience
120000
110000
100000
90000
80000
70000
60000
50000
40000
Salary
1
2
3
4
5
6
Years_Experience

Salary vs Experience
120000
110000
100000
90000
80000
70000
60000
50000
40000
Salary
1
2
3
4
5
6
Years_Experience

Years_Experience
Salary by Experience Level
120000
100000
80000
60000
40000
20000
0
Salary
Junior
Mid
Senior
Experience_Level